

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

Escola Politécnica - Engenharia de Software

Fundamentos da Teoria da Computação e Análise de Algoritmos - Professor José Guilherme Picolo

ANÁLISE EXPERIMENTAL DOS ALGORITMOS DE ORDENAÇÃO

1. Identificação e ambiente de execução

Campo	Informação
Nome do aluno	João Gabriel da Silva Leite
RA	24.787.392
Linguagem de programação utilizada	Python
Sistema operacional	MacOS
Processador	M1
Memória RAM	8gb
Link do GitHub com os códigos implementados	https://github.com/jgabrieldsl/lista10

2. Resultados experimentais

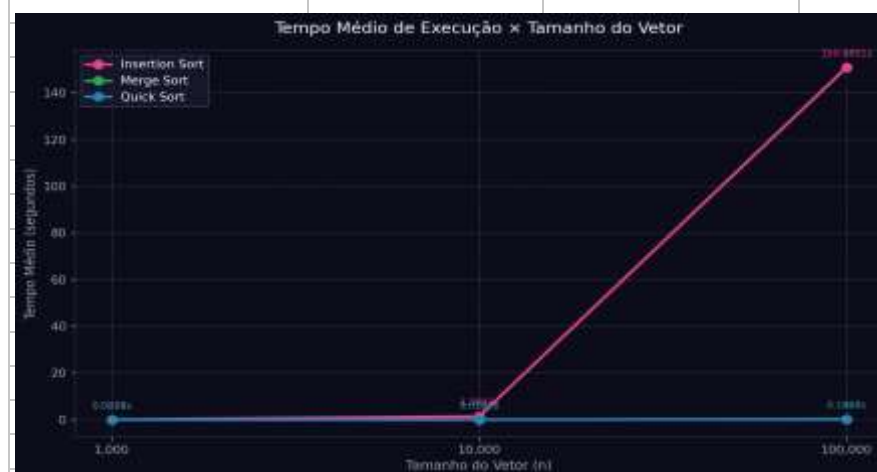
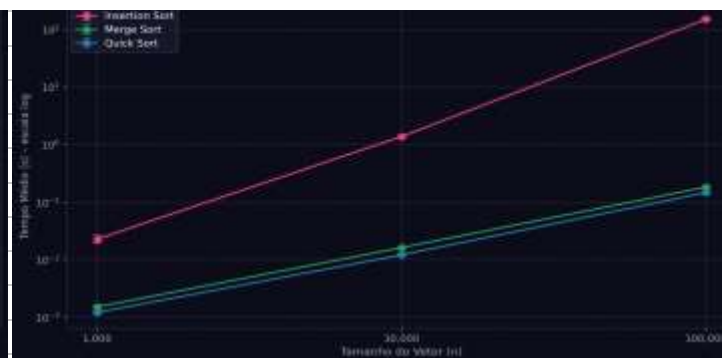
Algoritmo	Complexidade no caso médio	Tamanho do vetor	Execução 1 (s)	Execução 2 (s)	Execução 3 (s)	Tempo médio (s)	Desvio padrão (s)	Trocas ou movimentações
Insertion Sort	$O(n^2)$	1.000	0,03	0,02	0,02	0,02	0	238.485
Insertion Sort	$O(n^2)$	10.000	1,28	1,40	1,46	1,38	0,08	25.086.678
Insertion Sort	$O(n^2)$	100.000	148,63	155,65	148,35	150,88	3,38	2.499.413.009
Merge Sort	$O(n \log n)$	1.000	0	0	0	0	0	9.976
Merge Sort	$O(n \log n)$	10.000	0,02	0,02	0,02	0,02	0	133.616
Merge Sort	$O(n \log n)$	100.000	0,18	0,18	0,18	0,18	0	1.668.928
Quick Sort	$O(n \log n)$ médio	1.000	0	0	0	0	0	5.682
Quick Sort	$O(n^2)$ pior caso	1.000	0	0	0	0	0	5.682
Quick Sort	$O(n \log n)$ médio	10.000	0,01	0,01	0,01	0,01	0	78.419
Quick Sort	$O(n^2)$ pior caso	10.000	0,01	0,01	0,01	0,01	0	78.419
Quick Sort	$O(n \log n)$ médio	100.000	0,15	0,14	0,15	0,15	0,01	1.067.094
Quick Sort	$O(n^2)$ pior caso	100.000	0,15	0,14	0,15	0,15	0,01	1.067.094

Observação: Preencha os tempos das três execuções em segundos. Para algoritmos que não finalizarem em até 5 minutos, registre N/C nas execuções.

3. Gráfico comparativo

Comparação de Tempo Médio por Tamanho (Barras Agrupadas)

Tempo Médio (escala log-log) com Desvio Padrão



4. Análise dos resultados

Pergunta	Resposta
Os tempos obtidos ficaram coerentes com a complexidade teórica dos algoritmos? Explique.	Sim. Os resultados confirmaram a teoria com precisão: O Insertion Sort tem complexidade $O(n^2)$: quando n aumenta 10x, o tempo deveria crescer $\sim 100\times$. Na prática: $n=1k \rightarrow n=10k$: 0,0228s $\rightarrow$ 1,38s = $\sim 60\times$ $n=10k \rightarrow n=100k$: 1,38s $\rightarrow$ 150,88s = $\sim 109\times$ A média fica em torno de 100x, exatamente o previsto. O Merge Sort e o Quick Sort têm complexidade $O(n \log n)$: quando n aumenta 10x, o tempo deveria crescer ~ 10 a $15\times$ (pois $\log_{10}(10 \times n)$ cresce menos que linearmente). Na prática ambos cresceram ~ 10-$12\times$ a cada salto, confirmando o comportamento esperado.

A quantidade de trocas/movimentações acompanhou o crescimento do tempo de execução? Explique.	Sim, em geral acompanhou. O Insertion Sort realizou cerca de 2,5 bilhões de movimentações para $n=100.000$ e também foi de longe o mais lento. O Merge Sort e o Quick Sort fizeram respectivamente ~1,67 milhões e ~1,07 milhões de operações, sendo também proporcionalmente mais rápidos. Vale observar que o Merge Sort realizou mais operações que o Quick Sort, porém os tempos foram próximos, isso acontece porque o Merge Sort acessa a memória de forma sequencial e se beneficia do cache do processador, mostrando que o número de operações não é o único fator que determina o tempo de execução.
Qual algoritmo apresentou o melhor e o pior desempenho nos testes realizados?	Melhor desempenho: Quick Sort Foi o mais rápido nos três tamanhos testados: $n=1.000 \rightarrow 0,0012s$ $n=10.000 \rightarrow 0,012s$ $n=100.000 \rightarrow 0,147s$ Com pivô aleatório, comportou-se conforme $O(n \log n)$ médio e ainda realizou menos operações que o Merge Sort. Pior desempenho: Insertion Sort Foi catastrófico para entradas grandes: $n=1.000 \rightarrow 0,023s$ (ainda aceitável) $n=10.000 \rightarrow 1,38s$ (já lento) $n=100.000 \rightarrow 150,88s$ (~2,5 minutos) Para $n=100.000$, o Insertion Sort foi ~1.028× mais lento que o Quick Sort, demonstrando na prática o impacto devastador de uma complexidade $O(n^2)$ em entradas grandes.